

E-Com Project Interview Q&A

100 MNC-style questions with practical answers based on your Mercato full-stack project.

Area	Details
Frontend	React, Vite, React Router, reusable components, protected routes, loaders, empty states
Backend	Spring Boot REST APIs, layered controller-service-repository design
Security	JWT, BCrypt, Spring Security, role-based access for customer, seller, and admin
Database	JPA entities for users, products, sellers, cart, orders, wishlist, comments, OTP, reset tokens
Interview Focus	Architecture, APIs, auth, DB relations, business flows, debugging, testing, improvements

Project Overview and Architecture

1. Give me a quick overview of your E-Com project.

Answer: It is a full-stack e-commerce application named Mercato. The frontend is built with React and Vite, while the backend is a Spring Boot REST API. It supports customers, sellers, and admins. Customers can browse products, use cart and wishlist, place orders, comment, and rate products. Sellers can submit verification details and manage their products after approval. Admins can review seller requests and approve them.

2. Why did you separate the project into frontend and backend?

Answer: I separated them to keep UI logic and business logic independent. React handles pages, forms, state, routing, and user interaction. Spring Boot handles authentication, authorization, database operations, and business rules. This makes the project easier to maintain, test, and deploy.

3. What are the main modules in your project?

Answer: The main modules are authentication, product management, seller verification, admin dashboard, cart, orders, wishlist, comments and ratings, OTP verification, password reset, and profile management.

4. Which technologies did you use and why?

Answer: I used React and Vite for a fast frontend developer experience, Spring Boot for structured backend APIs, Spring Security with JWT for stateless authentication, Spring Data JPA for database access, MySQL-style relational persistence, Cloudinary for image uploads, and BCrypt for secure password hashing.

5. What is the role of the backend in this project?

Answer: The backend exposes REST APIs, validates authenticated users through JWT, applies role-based access, manages database entities through services and repositories, and contains the business logic for product listing, ordering, cart operations, wishlist, OTP, password reset, and seller approval.

6. What is the role of the frontend in this project?

Answer: The frontend provides the user experience. It has reusable components such as Navbar, ProductCard, Loader, Wishlist, Orders, CustomerForm, SellerForm, and pages for authentication, products, cart, seller dashboard, admin dashboard, product detail, profile, and reset password.

7. How would you explain the project flow to a non-technical person?

Answer: A user can register, log in, browse products, save products to wishlist, add items to cart, place orders, and manage their profile. A seller can request verification and, once approved by admin, add or update products. Admin controls seller approvals and user-related management.

8. What is the most important feature of your project?

Answer: The most important feature is the complete role-based e-commerce flow. It is not only product browsing; it includes customer actions, seller onboarding, admin approval, product management, cart/order checkout, wishlist, and authentication.

9. What makes your project different from a basic CRUD app?

Answer: It has multiple roles, secured APIs, real user workflows, relations between products, users, cart, orders, wishlist and comments, image upload integration, OTP/password reset flows, seller approval, and a frontend that reacts to empty states, loading states, and authenticated state.

10. If an interviewer asks the architecture, how will you answer?

Answer: I would say it follows a layered architecture. The React frontend calls backend REST endpoints. In the backend, controllers accept requests, services apply business logic, repositories talk to the database, and entities represent tables. Security is handled through a JWT filter and Spring Security configuration.

Frontend and User Interface

11. What frontend framework did you use?

Answer: I used React with Vite. React helped me build reusable UI components and Vite gave fast local development and optimized builds.

12. How is routing handled in the frontend?

Answer: Routing is handled with React Router. Pages such as home, products, product detail, cart, authentication, customer dashboard, seller dashboard, admin dashboard, order detail, and reset password are mapped to routes.

13. What is the purpose of ProtectedRoute?

Answer: ProtectedRoute checks whether the user has a valid token before allowing access to private pages. If the user is not authenticated or the token is expired, it redirects the user to the authentication page.

14. How does the frontend store authentication data?

Answer: The token is stored on the client side and reused in API calls. Helper functions read, validate, and remove the token when it is missing or expired.

15. How are API calls made from the frontend?

Answer: Some calls use a shared axios client and some use fetch directly. The API base URL points to the Spring Boot backend, and authenticated calls attach the JWT token in the Authorization header.

16. Why do you have separate API files like productApi, cartApi, and orderApi?

Answer: They group related API methods in one place, making components cleaner. Product-related calls stay in productApi, cart-related calls stay in cartApi, and order-related calls stay in orderApi.

17. What are reusable frontend components in your project?

Answer: Navbar, ProductCard, Loader, Wishlist, Orders, OrderSub, CartItem, CustomerForm, SellerForm, SellerReq, CommentBody, OTP input, Hero, About, and ContactUs are reusable pieces.

18. How do you show loading states in the frontend?

Answer: Loading state is managed with component state. A Loader component is rendered while data is being fetched or actions are being submitted, such as product loading, authentication, OTP, cart/order operations, seller products, and form submissions.

19. Why should a loader sometimes appear only inside a section instead of the whole page?

Answer: If only one section is loading, showing a full-screen loader blocks the whole interface unnecessarily. For example, product detail data should load inside the product-right area so the rest of the page does not jump or feel frozen.

20. How do you handle empty data on the frontend?

Answer: When the backend returns an empty array, the frontend shows friendly messages such as no products available, no orders available, or no wishlist items yet instead of showing a blank area or error state.

21. What issue did you fix in the wishlist empty state?

Answer: The frontend first showed the empty wishlist message but then treated a 400 response as a failed request. The fix was to make the backend return an empty list when no wishlist exists and make the frontend treat an empty wishlist response as a valid empty state.

22. How does the Navbar search work?

Answer: The search icon toggles the search input, and search text is used to call product search APIs or update products shown. The UI was adjusted so the nav links do not shift when the search input appears.

23. How do you prevent layout shifting in React UI?

Answer: I reserve stable space for elements, avoid changing layout width unexpectedly, use flex or grid consistently, and make dynamic content appear inside a fixed container when needed.

24. What is the ProductCard component responsible for?

Answer: ProductCard displays each product summary, including image, name, price, category, and navigation to product details. It helps avoid repeating product UI code across the app.

25. How do you handle image display in the frontend?

Answer: Images are displayed using backend image URLs or Cloudinary URLs. For users and products, the backend can redirect to stored image URLs, while uploads are first sent to Cloudinary.

26. How does the customer dashboard work?

Answer: CustomerDetail fetches the logged-in user, displays profile data, and switches between My Details, My Wishlist, and My Orders sections using component state.

27. How does the seller dashboard work?

Answer: SellerDash fetches the seller profile and seller verification details. If approved, it shows product management tools including added products and an update form.

28. How does the admin dashboard work?

Answer: AdminDash fetches pending seller requests and displays them. Admin can open seller details, view uploaded documents, and approve sellers.

Backend and REST APIs

29. What backend framework did you use?

Answer: I used Spring Boot. It provides a strong structure for REST controllers, services, repositories, configuration, security, and dependency injection.

30. What is the controller layer responsible for?

Answer: Controllers expose API endpoints, accept requests, extract path variables or request bodies, and call services. They should stay thin and avoid heavy business logic.

31. What is the service layer responsible for?

Answer: Services contain business logic. For example, ProductService maps product requests to entities, CartService updates carts and creates orders, OrderService reduces stock, and SellerService handles seller details.

32. What is the repository layer responsible for?

Answer: Repositories use Spring Data JPA to perform database operations. They abstract SQL-level details and expose methods such as findByEmail, findById, search, and find products by seller.

33. What are the main backend controllers?

Answer: AuthController, UserController, ProductController, SellerController, AdminController, CartController, OrderController, WishlistController, CommentController, OtpController, and PasswordResetController.

34. What APIs are public?

Answer: Public APIs include product listing and product detail, product search, authentication, OTP request/verify, password reset link, and reset password. Other APIs require a valid JWT token.

35. What APIs require authentication?

Answer: Profile, cart, wishlist, orders, seller dashboard actions, admin actions, product comments by user, product creation/update/delete, and role-specific routes require authentication.

36. How is product data managed?

Answer: ProductController exposes product APIs. ProductService creates, updates, deletes, searches, and fetches product data. ProductRepo handles persistence. ProductRequest and ProductResponse are used to keep API payloads separate from entity structure.

37. How does product search work?

Answer: The backend exposes a search endpoint that accepts a keyword and queries product-related fields. The frontend passes the search keyword and displays the matching products.

38. How is product availability calculated?

Answer: A product has stock quantity and status. When stock is available, status can be AVAILABLE. If stock reaches zero, it should become OUT_OF_STOCK.

39. How does the cart API work?

Answer: Authenticated users can get their cart, add or update cart items, delete cart items, delete the full cart, and place an order from the cart.

40. How does order placement from cart work?

Answer: CartService reads the user's cart, creates an order with order items, calculates total price, reduces product stock, saves the order, and clears the cart.

41. How does direct order placement work?

Answer: OrderService can place an order directly for a product without going through cart. It creates an order item, reduces stock, calculates total price, and saves the order.

42. How are comments and ratings handled?

Answer: CommentController exposes endpoints for product comments and user comments. CommentService saves comments with ratings, updates them, deletes them, and calculates average product rating.

43. How does wishlist functionality work?

Answer: Wishlist stores a user and a list of products. The API can check wishlist status for a product, fetch the user's wishlist, add a product, and remove a product.

44. Why did the wishlist 400 error happen?

Answer: The backend treated a missing wishlist as an error instead of a normal empty state. For a new user, no wishlist existing should mean an empty list, not a failed request.

45. How does seller approval work?

Answer: A seller submits business details and tax document URL. The seller details status starts as PENDING. Admin reviews seller requests and changes the status to APPROVED.

46. How does admin approve a seller?

Answer: AdminController exposes an approve endpoint. AdminService finds the seller details and updates the status, allowing the seller dashboard to show product management features.

Database and JPA

47. What type of database design does your project use?

Answer: It uses a relational design through JPA entities. Users, products, carts, orders, wishlist, comments, seller details, OTPs, and password reset records are represented as related tables.

48. What are the main entities?

Answer: User, Product, SellerDetails, Cart, CartItem, Order, OrderItem, Wishlist, Comment, Otp, and PasswordReset.

49. What is stored in the User entity?

Answer: User stores userId, name, email, password, role, phone/address-style profile data, and image URL. The role decides whether the user acts as customer, seller, or admin.

50. What roles exist in the project?

Answer: The project uses roles such as ADMIN, CUSTOMER, and SELLER. These roles are used by Spring Security for role-based access control.

51. What is stored in the Product entity?

Answer: Product stores product id, name, description, brand, price, category, stock quantity, status, image URL, and seller relation.

52. How is SellerDetails connected to User?

Answer: SellerDetails belongs to a seller user and stores verification data such as business description, tax ID, document image URL, and approval status.

53. How is Cart connected to User?

Answer: Each cart belongs to a user. Cart contains multiple CartItem records, and each cart item points to a product and stores quantity.

54. How is Order connected to User?

Answer: Each order belongs to a user and contains multiple OrderItem records. OrderItem stores product snapshot-style details such as product name, quantity, and price.

55. Why should OrderItem store price separately?

Answer: Because product price can change later. Storing price in OrderItem preserves the amount charged at the time of order.

56. How is Wishlist connected to Product?

Answer: Wishlist has a relationship with multiple products, often represented through a join table such as wishlist_products.

57. How are comments connected?

Answer: A comment belongs to a user and a product. It can store text and rating, allowing the product page to show reviews and average rating.

58. What is the purpose of Otp entity?

Answer: Otp stores temporary OTP data for email verification, including email, OTP value, and generated time or expiry-related data.

59. What is the purpose of PasswordReset entity?

Answer: PasswordReset stores password reset token information connected to an email or user so the backend can verify a reset link before changing the password.

60. Why use JPA repositories?

Answer: JPA repositories reduce boilerplate database code. They provide CRUD operations and allow custom finder methods based on method names or queries.

61. What is a DTO and why do you use it?

Answer: A DTO is a Data Transfer Object. ProductRequest, ProductResponse, OrderResponse, and similar objects prevent exposing entity internals directly and let the API send only required fields.

62. What is lazy loading in JPA?

Answer: Lazy loading means related data is fetched only when accessed. It can improve performance but must be handled carefully to avoid serialization or session issues.

Authentication and Security

63. How is authentication implemented?

Answer: Authentication uses login with email and password. Passwords are checked using BCrypt, and after successful login the backend generates a JWT token.

64. What does JWT contain in your project?

Answer: The JWT contains the user's email as subject and role as a claim. It also has an expiry time so old tokens cannot be used forever.

65. What is the token expiry?

Answer: The JWT service currently uses a short expiry window around 30 minutes. This reduces risk if a token is leaked.

66. How is JWT validated for each request?

Answer: A JWT filter intercepts requests, extracts the Bearer token, validates it, loads user details, and sets authentication in the Spring Security context.

67. What is BCrypt and why is it used?

Answer: BCrypt is a password hashing algorithm. It stores a secure hash instead of plain text passwords and includes a work factor to slow brute force attacks.

68. How does role-based authorization work?

Answer: Spring Security checks roles before allowing certain endpoints. For example, admin routes should require ADMIN, seller product routes should require SELLER, and customer flows require authenticated users.

69. What are the security strengths of your project?

Answer: It uses hashed passwords, stateless JWT authentication, role-based route protection, protected frontend routes, and authenticated API requests with Authorization headers.

70. What security risks still need improvement?

Answer: Secrets should not be hardcoded in application.properties, seller product update/delete should verify product ownership, reset tokens should expire, CORS should be restricted for production, and error messages should avoid leaking sensitive details.

71. Why is CSRF disabled?

Answer: CSRF is usually disabled for stateless JWT APIs because the token is sent in the Authorization header instead of relying on browser cookies. If cookies are used later, CSRF protection must be reconsidered.

72. How does CORS matter in your project?

Answer: The frontend and backend run on different origins during development, so CORS allows frontend requests. In production, CORS should allow only trusted frontend domains.

73. How would you protect admin APIs?

Answer: I would require ADMIN role, validate JWT on every request, avoid exposing sensitive data, add audit logs for approvals, and ensure admin actions cannot be triggered by normal users.

74. How would you protect seller product APIs?

Answer: I would require SELLER role and verify that the logged-in seller owns the product before allowing update or delete operations.

75. What is the difference between authentication and authorization?

Answer: Authentication verifies who the user is. Authorization decides what the authenticated user is allowed to do.

76. How is current user data fetched?

Answer: The frontend calls `auth/me` or `api/me` with the JWT token. The backend reads the authenticated principal and returns the current user's details.

77. How does password reset work?

Answer: The user requests a reset link, the backend generates a token, sends the link by email, and the reset page submits the new password with that token to update the password.

78. How should password reset be improved?

Answer: Reset tokens should have expiry, one-time-use behavior, secure random generation, and cleanup. The frontend should show clear loading and error states.

Business Flows

79. Explain the customer purchase flow.

Answer: A customer logs in, browses products, opens product details, adds products to cart or orders directly, confirms order, and then sees the order under My Orders.

80. Explain the cart flow.

Answer: The frontend sends product and quantity to the cart API. The backend creates a cart if needed, adds or updates the item, and returns the updated cart. Checkout converts cart items into an order.

81. Explain the wishlist flow.

Answer: A customer can check if a product is in wishlist, add it, remove it, and view all wishlisted products from the customer dashboard.

82. Explain the seller onboarding flow.

Answer: A seller logs in, submits business details and a tax certificate URL, waits for admin approval, and after approval can add and manage products.

83. Explain the admin seller approval flow.

Answer: Admin sees pending seller requests, opens seller details and document, verifies the information, and approves the seller. Approved sellers then get product features.

84. Explain product add/update flow for seller.

Answer: A verified seller opens seller dashboard, uses Add Product or product update form, uploads product image if needed, submits product details, and the backend saves or updates the product under that seller.

85. Explain direct order versus cart order.

Answer: Direct order creates an order for a selected product immediately. Cart order creates one order from all items currently in the user's cart.

86. How is stock updated when an order is placed?

Answer: The backend subtracts ordered quantity from product stock when an order is created. This should happen inside a transaction to prevent partial updates.

87. How should the app handle insufficient stock?

Answer: Before creating an order, the backend should compare requested quantity with stock. If not enough stock exists, it should return a clear error and not create the order.

88. How should cancelled or deleted orders affect stock?

Answer: If cancellation is supported, the backend should restore stock according to the order quantities. If delete only removes history, it should be restricted carefully.

89. How does rating affect product detail page?

Answer: The product detail page fetches product information, comments, and average rating. After a comment or rating change, it refetches or updates the displayed rating.

90. What happens when a new user has no cart, orders, or wishlist?

Answer: The frontend should show meaningful empty-state messages, and the backend should return empty arrays or an empty cart object instead of treating it as an error.

Testing, Debugging, and Deployment

91. How did you test the frontend?

Answer: I tested by running the frontend build and checking important pages manually. I also verified UI behavior such as loaders, empty states, search input, wishlist, seller dashboard, and forms.

92. How did you test the backend?

Answer: Backend can be tested through Maven tests and API tools. During local work, some compilation may depend on Maven dependency availability. Endpoint behavior can also be checked with Postman or browser fetch calls.

93. What would be your testing strategy?

Answer: I would add unit tests for services, integration tests for controllers with security, repository tests for queries, and frontend tests for important UI flows such as login, cart, wishlist, and seller product update.

94. What are important backend unit tests?

Answer: Cart checkout stock reduction, wishlist empty state, seller approval, seller product ownership, OTP expiry, password reset token validation, product search, and comment rating calculation.

95. What are important frontend tests?

Answer: Protected route redirects, login loader, OTP submit flow, empty wishlist/orders/products messages, seller product list rendering, update product form, navbar search behavior, and cart checkout states.

96. How would you debug a 400 Bad Request from the frontend?

Answer: I would check the network request URL, method, headers, body, token, backend controller mapping, service exception, and server logs. Then I would decide whether it is a real validation error or an empty-state case.

97. How would you debug a 401 Unauthorized?

Answer: I would inspect if the token exists, if it is expired, if the Authorization header is formatted as Bearer token, and whether the backend security rules allow that route.

98. How would you debug products not showing?

Answer: I would check the product API response, frontend state update, ProductCard rendering, category/search filters, backend ProductController route, and database product records.

99. How would you debug seller products not showing?

Answer: I would confirm seller approval status, seller user ID, the /seller/{sellerId}/products response, ProductService query, and frontend state rendering inside the seller product div.

100. How would you debug password reset link failure?

Answer: I would check email existence, reset token generation, mail sending config, backend exception logs, frontend response handling, and whether SMTP credentials or app password are valid.